

Apunte 42 — Cuando el programa no se cae sino se CUELGA: watchdogs y subprocessos

Proyecto Froth · aprender Machine Learning construyendo

1 El accidente real (madrugada del 14 de julio)

Lanzamos la re-cosecha sin techo de los 22 topics. El batch iba perfecto: 18 corpus gigantes (4.000–16.681 papers cada uno) en unas 3 horas... y de repente, silencio. A la mañana siguiente: **9 horas congelado** en el topic #20, sin error, sin mensaje, sin avanzar. El proceso seguía “vivo” (77 MB de RAM, cero CPU), esperando una respuesta de red que jamás llegó.

2 La lección central: un cuelgue NO es un crash

Nuestro `run_topic` ya era “a prueba de fallos”: envolvía todo el pipeline en `try/except`, y cada fallo se convertía en una fila **FAILED** del reporte para seguir con el siguiente topic. Sonaba blindado. No lo estaba.

Concepto: crash vs. cuelgue

Un **crash** lanza una excepción: `try/except` la atrapa y la vida sigue. Un **cuelgue** (hang) no lanza NADA: el programa se queda esperando para siempre (un socket muerto, un cómputo desbocado, un driver bloqueado). Ningún `try/except` del mundo interrumpe una espera infinita — para eso hace falta alguien MIRANDO DESDE AFUERA con un reloj: un **watchdog**.

Y como los topics corrían en serie, un solo cuelgue congeló el batch entero. La probabilidad no era despreciable: 22 topics $\times$ 4 APIs $\times$ miles de requests.

3 El arreglo: proceso hijo + reloj de pared

Cada topic corre ahora en un **subproceso** propio (`python -m froth.batch --one "<título>"`) vigilado por el padre con `subprocess.run(..., timeout=TOPIC_TIMEOUT_S)` (2.400 s, generoso: el topic legítimo más lento tardó 38 min). Si el hijo se pasa del reloj, Windows lo MATA, el padre escribe **FAILED: timed out** y sigue con el siguiente. El cuelgue pasó de “catástrofe silenciosa de 9 horas” a “una fila fea en el CSV”.

De paso acotamos los dos cómputos que podían desbocarse de verdad: el barrido DBCV para la granularidad automática es $\mathcal{O}(n^2)$ y se repite 18 veces — sobre 16.000 puntos son horas; ahora, pasado un tope (`DBCV_SUBSAMPLE_CAP = 5.000`) se barre sobre una submuestra aleatoria (la ELECCIÓN de granularidad es estable; el costo no), y el clustering final sí usa todos los puntos. Y el rellenado de abstracts, el único bucle de red sin presupuesto, recibió su límite de tiempo.

4 Bonus 1: el plan B de Beauvoir

El topic “Recovery of by-products of lithium from the a rare-metal granite” cosechó **0 papers**: sus términos derivados giraban sobre “a rare-metal granite”, demasiado nicho para las APIs. Arreglo: si la cosecha vuelve VACÍA, se reintenta UNA vez con una query amplia de alto recall (las primeras palabras de contenido del título: `recovery by-products lithium`). Resultado real: de 178 papers viejos a **4.937**.

5 Bonus 2: el enemigo externo (WDAC en olas)

La política corporativa de control de aplicaciones (WDAC) bloquea a ratos las DLL nativas de `scipy/hdbscan/torch` (“Una directiva de Control de aplicaciones bloqueó este archivo”). Medido en vivo: dejó trabajar 19 minutos a un hijo y 5 minutos después bloqueó a los otros dos en el import. Contra una política corporativa no se pelea: se ESPERA con paciencia automatizada — un sondeo barato (`python -c "import hdbscan, umap, torch"`) cada 5 minutos, y cuando la ola pasa, se lanza el trabajo pesado. El orquestador padre, además, solo importa `config` + `pandas`: sin DLLs científicas, WDAC no puede tumbarlo.

Ojo / error típico

Tres capas distintas de defensa, porque son tres fallos distintos: `try/except` atrapa *crashes*, el watchdog con timeout mata *cuelgues*, y el sondeo de ventanas espera *bloqueos del entorno*. Confundirlas es creer que un chaleco salvavidas te protege de un rayo.

En una frase

El batch se congeló 9 horas porque un cuelgue no lanza excepción y nada lo vigilaba. Arreglo en tres capas: subproceso por topic con timeout de pared (watchdog), cómputo acotado (DBCV por submuestra), y sondeo de ventanas WDAC. Resultado: 22/22 topics con estado claro y ~169.000 papers cosechados; Beauvoir pasó de 178 a 4.937 con el plan B de términos, a synthetic-ore topic de 237 a 9.723. Un sistema robusto no es el que nunca falla: es el que convierte cualquier fallo en una fila del reporte y sigue.